Lap Time Simulation Code User Guide

Raftar Formula Racing / Vehicle Simulation Workflow

1 Purpose

This document explains how to use the MATLAB lap time simulation workflow. The workflow consists of three main scripts:

1. `build_motor_map.m`
2. `lap_time_simulation_multilap_function.m`
3. `postprocess_lap_simulation_terminal_only.m`

The code estimates vehicle speed, acceleration, current draw, terminal power, accumulator heat loss, energy use, and lap time over one or more laps.

Important Vehicle and Component Assumption

This codebase is already configured for the current Raftar Formula Racing powertrain assumptions:

- Motor: **EMRAX 228 High Voltage**
- Cell: **Molicel P45B**
- Pack architecture used in the solver: **128 cells in series**

These components are used by Raftar and are also common choices among Indian Formula Student electric teams. If another team uses a different motor, cell, final drive ratio, pack architecture, or current–torque behaviour, the relevant lookup tables, limits, and fitted equations must be updated before using the results for design decisions.

2 Required Files

Place the following files in the same MATLAB working folder:

- `build_motor_map.m`
- `lap_time_simulation_multilap_function.m`
- `postprocess_lap_simulation_terminal_only.m`
- `fsg_track_left_right_data.xlsx`

The solver expects the track file to contain three columns:

Column	Meaning	Example
1	Direction	`Left`, `Right`, `Straight`
2	Arc length [m]	`12.5`
3	Radius [m]	`25.0`

Very large radius values are automatically treated as straights.

3 Overall Workflow

Run the scripts in this order:

```
build_motor_map
lap_time_simulation_multilap_function
postprocess_lap_simulation_terminal_only
```

The first script builds the motor torque map. The second script runs the lap simulation and saves `lap_solver_log.mat`. The third script loads this log and creates plots.

4 Step 1: Build the Motor Torque Map

Run:

```
build_motor_map
```

This creates:

```
emrax228_hvcc_torque_map.mat
```

The file contains:

- `voltage_points`
- `rpm_points`
- `torque_map`
- `motor`
- `limits`
- `map_info`

The torque map gives the maximum available motor torque as a function of DC bus voltage and motor speed:

$$T_{\max} = f(V_{\mathrm{dc}}, \mathrm{rpm})$$

Before running the lap solver, load this map into the workspace:

```
load("emrax228_hvcc_torque_map.mat")
```

The lap solver requires `rpm_points`, `voltage_points`, and `torque_map` to already exist in the MATLAB workspace.

5 Step 2: Configure the Lap Solver

Open:

```
lap_time_simulation_multilap_function.m
```

Most user-editable values are grouped at the top under:

```
%% Constants, lookup tables, and tuning parameters
```

5.1 Set Number of Laps

To run one lap:

```
simulation.num_laps = 1;
```

To run five laps:

```
simulation.num_laps = 5;
```

The solver runs the same single-lap track repeatedly. The ending SoC of one lap is passed as the starting SoC of the next lap.

5.2 Set Vehicle Parameters

Vehicle parameters are defined in this block:

```
vehicle.tyre_radius = 229 / 1000;       % [m]
vehicle.fdr = 4.1;                      % final drive ratio [-]
vehicle.mass = 270;                     % [kg]
vehicle.frontal_area = 1.2;             % [m^2]
vehicle.front_load_dist = 0.45;         % [-]
vehicle.front_Aero_dist = 0.5;          % [-]
vehicle.driven_wheels = 2;
```

Change these values if the vehicle mass, tyre radius, final drive ratio, or aero geometry changes.

5.3 Set Aero and DRS Parameters

Default aero coefficients are:

```
aero.default.Cd = 1.2;
aero.default.Cl = 2.2;
aero.rolling_resistance_coeff = 0.05;
```

DRS coefficients are:

```
aero.drs.Cd = 0.1488;
aero.drs.Cl = 0.5963;
```

DRS zones are defined using track mesh indices:

```
aero.drs_zones = [
    30,   85;
    500,  580;
    700,  750;
    1155, 1225
];
```

To disable DRS:

```
aero.enable_drs = false;
```

To enable DRS:

```
aero.enable_drs = true;
```

5.4 Set Battery Parameters

Battery and accumulator parameters are:

```
battery.max_pack_voltage_V = 537.6;
battery.power_limit_W = 80000;
battery.internal_resistance_ohm = 0.8;
battery.capacity_Ah = 13.5;
battery.initial_SoC = 1.0;
```

The model computes battery terminal power as:

$$P_{\text{terminal}} = (V - IR_{\text{int}})\, I$$

Accumulator heat loss is calculated as:

$$P_{\text{loss}} = I^2 R_{\text{int}}$$

5.5 Set Voltage Lookup Table

The solver uses a lookup table for pack open-circuit voltage:

```
voltageLUT.soc_points = [0.00; 0.10; ... ; 1.00];

voltageLUT.cell_voltage_points = [2.50; 3.20; ... ; 4.20];

voltageLUT.series_cells = 128;
```

The pack voltage is calculated as:

$$V_{\text{pack}} = V_{\text{cell}}(\text{SoC}) \times N_{\text{series}}$$

Replace the cell voltage values with measured cell OCV data if available.

5.6 Set Motor Torque Limit

An additional motor torque cap is applied inside the solver:

```
driveLimits.motor_torque_limit_Nm = 50;
```

This prevents the interpolated torque map from exceeding the chosen motor torque limit.

6 DC Current Estimation Equation

The function `getdccurrent()` estimates DC current from pack voltage, motor speed, and motor torque. This is not a generic EM model or a universal motor controller equation. It is a fitted polynomial equation derived from testing and data generated for the **RFR25** vehicle configuration, and it should be used only for the **EMRAX 228** setup unless it is re-fitted using new test data.

Inside the function, the variables are defined as:

$$x = V_{\text{pack}}$$

$$y = \text{motor speed in rpm}$$

$$z = T_{\text{motor}}$$

The motor speed is calculated from vehicle speed as:

$$y = \frac{v \times 60}{2\pi r_{\text{tyre}}} \times \text{FDR}$$

The motor torque is approximated from longitudinal acceleration as:

$$z = \frac{mar_{\text{tyre}}}{\text{FDR}}$$

The fitted current equation used in the code is:

$$\begin{aligned} I_{\text{dc}} = {} & 30.8902266940 - 0.2676776326x + 0.0045047646y + 0.0518698998z \\ & + 0.0007522720x^2 - 0.0000407406xy - 0.0009983754xz \\ & + 0.0000011995y^2 + 0.0007925304yz + 0.0044117065z^2 \\ & - 0.0000006752x^3 + 0.0000000589x^2y + 0.0000017863x^2z \\ & - 0.0000000005xy^2 - 0.0000010541xyz - 0.0000080998xz^2 \\ & - 0.0000000002y^3 - 0.0000000132y^2z - 0.0000001955yz^2 \\ & - 0.0000167599z^3 \end{aligned}$$

The output is then made positive for traction-current estimation in the MATLAB implementation.

When This Equation Must Be Changed

This fitted current equation should be updated if any of the following are changed:

- Motor model other than EMRAX 228
- Motor controller or inverter
- Current limit strategy
- Gear ratio or tyre radius
- Torque command strategy
- Vehicle for which the current data was collected

For a different powertrain, the correct approach is to collect or simulate current data over voltage, speed, and torque operating points, then fit a new equation or replace this function with a validated lookup table.

7 Step 3: Run the Lap Solver

After loading the motor map, run:

```
lap_time_simulation_multilap_function
```

The solver will:

1. Generate the GGV map.
2. Read and process the track file.
3. Generate the track coordinates.

4. Precompute DRS-dependent aero coefficients.
5. Run the single-lap solver repeatedly for `simulation.num_laps`.
6. Carry final SoC from one lap to the next.
7. Save all results in `lap_solver_log.mat`.

8 How Multi-Lap Simulation Works

The main multi-lap loop is:

```
current_SoC = battery.initial_SoC;

for lap_idx = 1:num_laps

    [lapLogs{lap_idx}, current_SoC] = solveSingleLap( ...
        current_SoC, ...
        lap_idx, ...
        distance_offset_m, ...
        time_offset_s, ...
        ctx);

end
```

The important idea is:

$$\mathrm{SoC}_{\mathrm{start},\,k+1} = \mathrm{SoC}_{\mathrm{end},\,k}$$

So lap 2 starts from the final SoC of lap 1, lap 3 starts from the final SoC of lap 2, and so on.

9 Solver Output

The solver saves:

```
lap_solver_log.mat
```

This file contains:

```
solverLog
```

Important fields include:

Field	Meaning
`distance_m`	Cumulative distance [m]
`time_s`	Cumulative time [s]
`lap_index`	Lap number for each logged point
`final_speed_ms`	Final vehicle speed [m/s]
`accel_ms2`	Longitudinal acceleration [m/s^2]
`SoC`	State of charge [-]
`voltage_V`	Pack voltage [V]
`current_A`	DC current [A]
`terminal_power_W`	Battery terminal power [W]
`heat_loss_W`	Accumulator I^2R heat loss [W]
`accumulator_heat_loss_W`	Same heat loss, clearer name [W]
`lap_summary`	Per-lap time, SoC use, and index range

10 Step 4: Post-Process Results

After running the solver, run:

```
postprocess_lap_simulation_terminal_only
```

This script loads `lap_solver_log.mat` and plots:

- Track map coloured by speed
- Speed, acceleration, SoC, voltage, current, and terminal power vs distance
- GGV envelope
- Terminal power heat map
- Summary statistics
- Electrical logs vs time
- Cumulative terminal energy and accumulator heat loss energy

It also exports a processed structure to the MATLAB base workspace:

```
post
```

11 Recommended Usage Order

A typical MATLAB session should look like this:

```
clear;
clc;
close all;

build_motor_map
load("emrax228_hvcc_torque_map.mat")

lap_time_simulation_multilap_function

postprocess_lap_simulation_terminal_only
```

12 Common Edits

12.1 Changing Number of Laps

Edit:

```
simulation.num_laps = 5;
```

12.2 Changing Initial SoC

Edit:

```
battery.initial_SoC = 1.0;
```

For 90% initial SoC:

```
battery.initial_SoC = 0.9;
```

12.3 Changing Pack Resistance

Edit:

```
battery.internal_resistance_ohm = 0.8;
```

12.4 Changing Power Limit

Edit:

```
battery.power_limit_W = 80000;
```

12.5 Changing Motor Torque Limit

Edit:

```
driveLimits.motor_torque_limit_Nm = 50;
```

13 Important Notes

- Run `build_motor_map.m` before the solver, or manually load an existing motor map.
- The solver requires `rpm_points`, `voltage_points`, and `torque_map` in the workspace.
- The track file must be available in the current MATLAB folder.
- Multi-lap simulation does not duplicate the track file. It repeatedly calls the single-lap solver and appends the logs.
- The final SoC of one lap is automatically used as the initial SoC of the next lap.
- The post-processing script should be run only after `lap_solver_log.mat` has been generated.